Lab 6: Code and Branch Coverage Analysis Report

Name: Akshata Amara
SRN: PES1UG24AM025

1. Objective

The objective of this lab is to design test cases for the train_fare.py module (functions: base_fare, concession, tatkal_charge) that achieve, first, complete statement (code) coverage, and then complete branch coverage, and to analyze the differences between the two coverage criteria.

2. Module Under Test

train_fare.py contains three functions:
- base_fare(travel_class, distance_km): computes the base fare for a class/distance, validates the class and distance, and enforces a minimum fare of 50.
- concession(age, gender, is_tatkal): computes a fare concession based on tatkal status, age bracket, and gender.
- tatkal_charge(travel_class, fare): computes the tatkal surcharge, clamped between 100 and 400.

Two separate test files were used to keep each coverage goal independently verifiable: CodeCoverageTest_AM025_application.py (statement coverage) and BranchCoverageTest_AM025_application.py (branch coverage).

3. Code (Statement) Coverage

Test cases were first written in CodeCoverageTest_AM025_application.py to execute every statement in train_fare.py at least once, without regard to which direction each condition resolved. Command used:

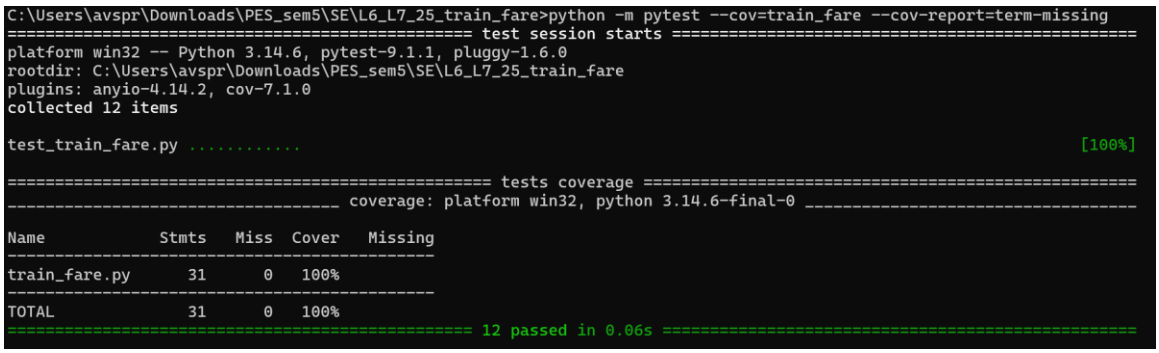
```
python -m pytest CodeCoverageTest_AM025_application.py --cov=train_fare
--cov-report=term-missing
```

Test cases designed for statement coverage:

Test Case	Input	Purpose (line reached)
test_sleeper_fare	base_fare("SL", 500)	Normal fare computation, no

		minimum-fare bump
test_base_fare_unknown_class	base_fare("XY", 100)	raise ValueError for unknown class
test_base_fare_invalid_distance	base_fare("SL", 0)	raise ValueError for invalid distance
test_base_fare_minimum_bump	base_fare("SL", 10)	fare < 50 bump to minimum fare line
test_adult_no_concession	concession(30,"M",False)	default return 0 line
test_concession_tatkal	concession(30,"M",True)	is_tatkal early return line
test_concession_infant	concession(3,"M",False)	age < 5 return line
test_concession_child	concession(10,"M",False)	5 <= age <= 12 return line
test_concession_senior_female	concession(65,"F",False)	age >= 60 and gender == F line
test_concession_senior_male	concession(65,"M",False)	age >= 60, non-female return line
test_tatkal_charge_non_sl_clamped_high	tatkal_charge("2A",2000)	non-SL charge formula + charge>400 clamp line
test_tatkal_charge_sl_clamped_low	tatkal_charge("SL",500)	SL charge formula + charge<100 clamp line

Result: 12 tests passed, 100% statement coverage on train_fare.py (31/31 statements), exceeding the 90% requirement.



4. Branch Coverage

Running the Phase 1 suite with branch tracking enabled (--cov-branch) showed 98% branch coverage with one partial branch flagged: 36->38 in tatkal_charge. This is the elif charge < 100 condition evaluating False — i.e., the case where the computed surcharge already falls within the allowed [100, 400] range and no clamp is applied. Every individual statement in

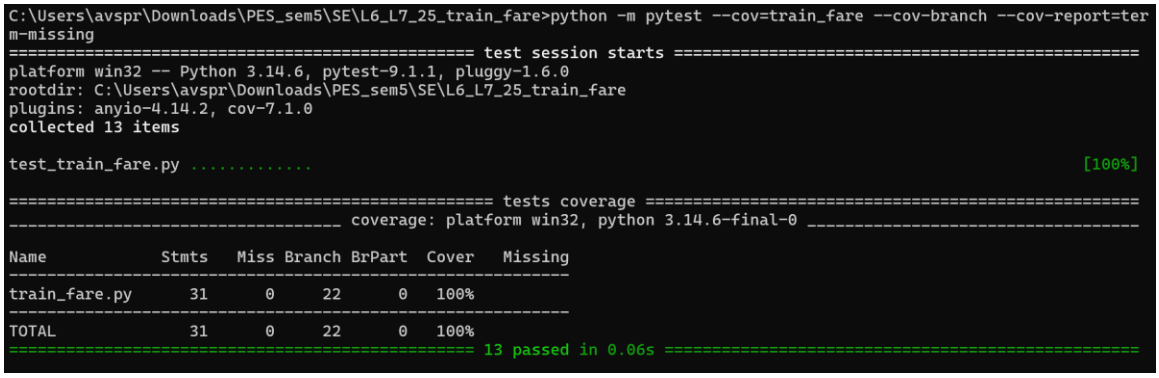
the function had already executed, but this particular control-flow edge had never been taken, so statement coverage alone could not detect the gap. A dedicated file, BranchCoverageTest_AM025_application.py, was created so that branch coverage can be verified standalone — it carries all test cases needed to exercise every True/False outcome of every if/elif in train_fare.py, including the missing pass-through case.

```
python -m pytest BranchCoverageTest_AM025_application.py --
cov=train_fare --cov-branch --cov-report=term-missing
```

Test case added (beyond Phase 1) to close the branch gap:

Test Case	Input	Branch exercised
test_tatkal_charge_no_clamp	tatkal_charge("SL", 2000) -> charge = 200	if charge > 400: False, elif charge < 100: False (pass-through, no clamp)

Result: 13 tests passed, 100% statement coverage and 100% branch coverage (22/22 branches).



5. Analysis: Code Coverage vs. Branch Coverage

Code (statement) coverage only verifies that a line of code was reached at least once; it does not check which way a decision resolved. Branch coverage additionally requires that every True/False outcome of every if/elif condition be exercised, i.e. every edge of the control-flow graph, not just every node.

In this lab, 100% statement coverage was reached with 12 test cases, but branch coverage was only 98% with the same suite — the pass-through branch of the clamp logic in tatkal_charge (charge already between 100 and 400) was never tested, even though every line in the function had executed. This demonstrates the key relationship between the two criteria: branch coverage subsumes statement coverage (100% branch coverage guarantees 100% statement coverage for reachable code), but the reverse is not true — statement coverage can be complete while branch coverage still has gaps.

Only one additional test case (test_tatkal_charge_no_clamp) was required to close this gap and reach 100% branch coverage, because all other decision points in base_fare and concession already had both outcomes exercised by the original statement-coverage suite.

6. Conclusion

Statement coverage is a necessary but insufficient criterion for thoroughly testing decision-heavy code. Branch coverage caught a logical path (the no-clamp case in tatkal_charge) that statement coverage missed entirely, despite both using the same underlying code. For modules with non-trivial conditional logic, branch coverage should be the minimum bar for confidence in correctness.